Ad Soyad: _____	Tarih: _____
Bölüm 13 – Ünite 1 – *OBYS Sunucu Tasarımı – Çalışma Kağıdı*	

Süre

Bu çalışma kağıdını tamamlamak için tahmini süre: **35–45 dakika**.

Konu Hatırlatma

Bu ünitede çok istemcili OBYS sunucusunun **beş katmanı** (Listener, Session, Router, Service, Repository), **bağlantı yaşam döngüsü** ve **thread-per-connection** desenini tanıdık. Detaylar için: `sec:13_u1_sunucu_mimari`, `sec:13_u1_baglanti_yonetimi`, `sec:13_u1_istemci_basi_thread`.

Görev 1: Sunucu Mimarisi – Katman Eşleştirme

(6 dakika – 18 puan)

G1: Beş Katmanı Sorumluluğuyla Eşleştir

Aşağıdaki tabloda OBYS sunucusunun beş mimari katmanı solda, sorumlulukları sağda karışık verilmiştir. Her katmanın yanına doğru harfi yazın.

#	Katman	Eşleşme	Harf	Sorumluluk
1	Listener (Dinleyici)	_____	A	İş mantığı: “Öğrenci ekle”, “Not güncelle” gibi yüksek seviye işlemleri yürütür
2	Session (Oturum)	_____	B	JSON mesajındaki `komut` alanına bakıp doğru servise yönlendirir (dispatcher)
3	Request Router	_____	C	Belirli IP/port’ta TCP bağlantılarını `accept` eder; veri okumaz, iş yapmaz
4	Service (Servis)	_____	D	Verinin kalıcı saklandığı yer; `shared_mutex` korumalı `Depo` burada
5	Repository (Depo)	_____	E	Bir istemcinin bağlantı süresince yaşayan durumu (kullanıcı, yetki, jeton)

İpucu

Her isteğin yolu sol üstten sağ alta dökülür: *Listener* → *Session* → *Router* → *Service* → *Repository*. Bir katmanın adı genelde sorumluluğunu da hatırlatır.

Görev 2: Thread-per-Connection ve C10K

(8 dakika – 22 puan)

G2: Üç Senaryoyu Yorumla

Linux’ta her thread varsayılan **8 MB sanal stack** alanı kullanır. Aşağıdaki üç senaryo için (a) toplam bellek tahminini hesaplayın, (b) ne olur kısaca yazın.

Senaryo 1. **200 eş zamanlı bağlantı** (tipik OBYS ders kayıt günü).
Toplam stack tahmini: _____
Ne olur? _____

_____

Senaryo 2. **2.000 eş zamanlı bağlantı** (orta ölçek üniversite, dönem başı).
Toplam stack tahmini: _____
Ne olur? _____

_____

Senaryo 3. **10.000 eş zamanlı bağlantı** (klasik C10K problemi).
Toplam stack tahmini: _____
Ne olur? _____

_____

Soru: Hangi senaryoda thread havuzuna ya da event loop'a geçmek *zorunlu*dur ve neden?

_____

_____

Görev 3: Hata Bulma – ClientHandler Bellek Sızıntısı

(8 dakika – 22 puan)

G3: Aşağıdaki Kodda En Az 3 Sorun Var

Bu `incomingConnection` kalıbında bağlantı kapandıktan sonra ne soket ne de handler temizleniyor. Kodu inceleyin, sorunları işaretleyin.

```
void OBYSSunucu::incomingConnection(qintptr soketDescriptor) {
    QTcpSocket* soket = new QTcpSocket();              // !!! parent yok
    soket->setSocketDescriptor(soketDescriptor);

    QThread* worker = new QThread();                   // !!! parent yok
    ClientHandler* handler = new ClientHandler(soket); // !!! handler
        nereye taşındı?

    worker->start();

    // Bağlantı kapandığında ne olur? Hiçbir temizlik yok.
    // Soket parent'siz, handler parent'siz, worker parent'siz.
}
```

Sorun 1: _____

_____

Sorun 2: _____

_____

Sorun 3: _____

_____

En kritik tek satırlık düzeltme (handler ve soket temizliği için):

```
connect(soket, &QTcpSocket::disconnected, _____, _____);
```

Görev 4: `incomingConnection` Override – Eksikleri Doldur (8 dakika – 20 puan)

G4: Worker Thread'i Doğru Kur

Aşağıdaki iskelette **beş boşluk** var. `soket`, `handler` ve `worker` arasındaki sahiplik ve thread affinity zincirini kurun.

```
void OBYSSunucu::incomingConnection(qintptr soketDescriptor) {
    if (aktif_baglanti_.load() >= MAX_BAGLANTI) {
        // Limit asildi
        QTcpSocket reddet;
        reddet.setSocketDescriptor(soketDescriptor);
        reddet.disconnectFromHost();
        return;
    }

    QThread* worker = new QThread(________);    // (1) parent
    ClientHandler* handler =
        new ClientHandler(soketDescriptor, &yonlendirici_);

    // (2) handler'i worker thread'ine tasi:
    handler->moveToThread(________);

    // (3) thread basladiginda handler->run cagrilsin:
    connect(worker, &QThread::started,
            handler, ________);

    // (4) handler bitince worker'i durdur:
    connect(handler, &ClientHandler::baglantiKapandi,
            worker, ________);

    // (5) worker bitince hem worker hem handler silinsin:
    connect(worker, &QThread::finished,
            worker, &QObject::deleteLater);
    connect(worker, &QThread::finished,
            handler, ________);

    worker->start();
    aktif_baglanti_.fetch_add(1);
}
```

İpucu

Qt'de bir `QObject` **yaratıldığı thread'e aittir** (thread affinity). Soketi `handler`'in içinde yeni thread'de kurmak gerekir; `moveToThread` sahibinin event loop'unu değiştirir. `deleteLater` cross-thread güvenli silmedir.

Görev 5: BONUS – Kötü Davranan İstemciyi Bantlama (7 dakika – 18 puan)

G5: Üç Davranışı Tespit Edip Yorumla

Sunucu, "kötü davranan istemciyi" tespit edip bantlamak (ban) ister. Aşağıdaki üç davranışın *neden şüpheli* olduğunu ve sunucunun ne tür bir önlem alabileceğini yazın.

Davranış 1: İstemci bağlandı ama 30 saniye boyunca tek bir bayt göndermedi (slowloris benzeri).
Neden şüpheli? _____
Önlem: _____

Davranış 2: İstemci son 5 saniyede 200 farklı kullanıcı adı ile başarısız `LOGIN` denemesi yaptı.
Neden şüpheli? _____
Önlem: _____

Davranış 3: İstemci length-prefix alanında `2 GB` iddia eden bir mesaj başlattı.
Neden şüpheli? _____
Önlem: _____

Tartışma Soruları (sınıf içi açık uçlu)

T1. Thread-per-connection ile thread havuzu arasındaki seçim, OBYS'nin *kullanıcı sayısına* ne kadar bağlıdır? Sınıfta hangi eşik değerlerini birlikte tartışırsınız?

T2. "Bir istemcinin yarattığı sorun yalnızca kendi thread'ini etkiler" iddiası tamamen doğru mu? Hangi durumlarda bir istemcinin davranışı diğer istemcileri de etkiler?

T3. Audit log'u "yardımcı katman" olarak çizdik. Bu yardımcının da `Depo` kadar tasarımı önemsenmeli mi? Niçin?

T4. OBYS sunucusunda `SIGTERM` geldiğinde "düzgün kapanış" (graceful shutdown) ne kadar sürede tamamlanmalı? Eş zamanlı 200 istemci varsa nasıl ilerlersin?